3D Map Generation from Perlin Noise

Dante Candia (F11215109)
Computer Graphics Final Project

***Abstract*—This work describes a C++ program for real-time 3D terrain generation using a custom Perlin-noise library (`perlin.hpp`) and OpenGL with ImGui for parameter control. The noise library implements Perlin noise, fractal (multi-octave) noise, ridged noise, terracing, and an island mask. The main program constructs a mesh by sampling elevation and moisture, classifies biomes per vertex, uploads vertex/normal/color data to OpenGL buffers, and renders either a shaded or geometry-shader wireframe terrain. ImGui sliders trigger mesh rebuilds on parameter changes, and simple snowfall/rainfall particle systems overlay weather effects.**

I. Introduction

Procedural terrain generation (PTG) streamlines the creation of expansive, varied virtual landscapes by relying on algorithmic techniques rather than manually authored heightmaps [2]. Natural terrains exhibit fractal characteristics—self-similar patterns across scales—which classic Euclidean primitives cannot reproduce effectively [2], [1]. Noise functions such as Value Noise and Perlin Noise provide structured pseudo-random outputs that, when combined via fractional Brownian motion (fBm), yield realistic, multi-scale heightfields [2], [3]. Our implementation builds on these principles by generating gradient-noise samples, aggregating multiple octaves, and optionally applying ridged transformations, terracing, and radial island masks to sculpt peaks, plateaus, and coastal falloff [1], [5].

Beyond noise-based heightmaps, other PTG approaches include software agents, evolutionary algorithms, and physically based erosion simulations, each adding control or realism in distinct ways [2], [7]. However, for real-time interactive applications, multi-octave gradient noise remains a preferred choice due to its balance of performance, memory footprint, and visual quality [2], [1]. This project's objective was to implement a custom 2D Perlin-noise library in C++, integrate it into an OpenGL/ImGui framework, and allow users to adjust parameters such as noise scale, octaves, persistence, ridged blend, terrace steps, and biome thresholds at runtime. The result is a stand-alone application that dynamically rebuilds a 3D mesh with per-vertex normals and colors, classifies biome regions based on elevation and moisture sampling, and optionally toggles a geometry-shader wireframe ripple or simple snowfall/rainfall particle effects.

II. Custom Perlin-Noise Library

A. Gradient Noise (Perlin 2D)

Perlin noise assigns a pseudo-random unit gradient at each integer lattice point and interpolates dot-products between those gradients and the sample point. In `perlin.hpp`, the function

```
glm::vec2 RandomGradient(int ix, int iy);
```

uses hashing of `ix`, `iy`, and a constant `SEED` to produce a repeatable 2D unit vector. For a sample point (x, y), each lattice corner (i, j) computes:

$$\text{DotGridGradient}(i, j; x, y) \;=\; \mathbf{g}_{ij} \cdot (x - i,\; y - j),$$

where $\mathbf{g}_{ij} = \text{RandomGradient}(i, j)$. The interpolation weight in each dimension is computed by the quintic fade:

$$S(t) = 6t^5 - 15t^4 + 10t^3, \quad t \in [0, 1].$$

The four corner dot-products are then bilinearly interpolated using $S(t)$. This process produces a smoothly varying noise value in $[-1, 1]$, which is later remapped to $[0, 1]$.

B. Fractal (Multi-Octave) Noise

To add detail across scales, multiple octaves of Perlin noise are summed. The function

```
float fractalNoise2d(float x, float y,
                     int octaves, float persistence);
```

computes:

$$\text{fBM}(x, y) \;=\; \sum_{i=0}^{\text{octaves}-1} \big(\text{persistence}^i \times \text{SampleNoise}(x \cdot 2^i,\; y \cdot 2^i)\big).$$

Since SampleNoise $\in [-1, 1]$, the sum is remapped to $[0, 1]$ by multiplying by 0.5 and adding 0.5.

C. Ridged Noise

Ridged noise emphasizes peaks by folding the absolute value of each octave's Perlin output. The function

```
float ridgedNoise(float x, float y);
```

evaluates:

$$r_i = 1 - \big|\text{SampleNoise}(x \cdot 2^i,\; y \cdot 2^i)\big|, \quad r = \sum_{i=0}^{\text{octaves}-1} \big(\text{persistence}^i \, r_i\big).$$

A final blend between fractal (f) and ridged (r) components is controlled by `ridgedMix`:

$$e = f \times (1 - \text{ridgedMix}) \;+\; r \times \text{ridgedMix}.$$

D. Terracing and Island Mask

Terracing quantizes a continuous elevation value into discrete bands:

```
float terrace(float height, int steps);
```

which computes

$$e_{\text{terraced}} = \frac{\lfloor e \times \text{steps} \rfloor}{\text{steps}}.$$

The island mask applies a radial falloff so that edge values taper toward zero:

```
float radialMask(float x, float z) {
   return 1.0f - \sqrt{x^2 + z^2};
}
float applyIsland(float elevation, float mask) {
   return elevation \times mask;
}
```

Multiplying the raw elevation by mask(x, z) yields an island shape where heights at the edges go to zero.

III. Main Program

A. Initialization and Callbacks

GLFW is first initialized, an OpenGL 3.3 core-profile context is requested, and a window of size 1024×768 is created. GLAD loads all function pointers. Depth testing and program-point-size are enabled to allow proper 3D rendering and point-sprite sizing. Global variables track camera state:

`yaw`, `pitch`, `distanceToCenter`

which affect the view matrix. Mouse callbacks are registered so that on left-mouse button and drag, `yaw` and `pitch` are updated, and on scroll, `distanceToCenter` is adjusted. However, before applying any camera update, each callback checks:

```
if (ImGui::GetIO().WantCaptureMouse) return;
```

This ensures that when the user is interacting with ImGui widgets, camera rotation and zoom are disabled.

ImGui is initialized in three steps:

1) `ImGui::CreateContext()` to set up the core ImGui state.
2) `ImGuiImplGlfwInitForOpenGL(window, true)` to connect ImGui to GLFW.
3) `ImGui_ImplOpenGL3_Init` to prepare the OpenGL3 backend.

ImGui's style is set to Dark with `ImGui::StyleColorsDark()`.

B. Shader Compilation

Two helper routines wrap GLSL compilation and linking:

- `compileShader(type, source)` creates a shader of the given type, loads the source, compiles, and checks for errors.
- `createProgram(vertexSource, fragmentSource)` calls `compileShader` on each stage, attaches both to a new program, links, checks for errors, then deletes the intermediate shader objects.

Using these, three programs are constructed:

- **terrainShader**: Vertex and fragment shaders implement basic diffuse shading by computing the dot product of the interpolated normal and a constant light direction.
- **particleShader**: A vertex shader emits points at the particle positions and sets `gl_PointSize`. The fragment shader colors these points uniformly (white for snow, blue for rain).
- **geoProgram**: A vertex shader applies a per-vertex vertical offset of the form

  $$\Delta y = A \sin\big(\text{frequency} \sqrt{x^2 + z^2} \; - \; uTime \times \text{speed}\big),$$

  creating a circular ripple. A geometry shader then takes each input triangle and emits its three edges as line strips, producing a purple wireframe in the fragment stage.

C. Terrain Mesh Generation

Whenever any terrain parameter changes (e.g., grid size, noise scale, octaves, etc.), a global `dirty` flag is set to `true`. At the start of each frame, the code checks:

```
if (dirty) {
   buildTerrain()
   dirty = false
}
```

The `buildTerrain()` routine constructs a regular $(N + 1) \times (N + 1)$ grid of vertices, where $N =$ `gridSize`. Each vertex's x, z position is mapped from $[-1, 1]$ based on its grid index. Its elevation e is computed by calling `getElevation(x,z)`, which blends fractal and ridged noise, then optionally applies terracing and an island mask. Moisture m is sampled independently via a separate fractal noise call. A biome classification function maps (e, m) to a color per vertex. All vertices are stored in a CPU-side array along with an initial normal of $(0, 0, 0)$.

Next, two triangles are generated per grid cell, producing an index list of size $2N^2$. After indexing, the code iterates over every triangle, computes its face normal via a cross product, and accumulates this normal into each of the three vertex normals. Finally, each vertex normal is normalized. The vertex positions, normals, and colors are then uploaded to a GPU VBO, and the index list is uploaded to an EBO. A VAO is configured with three attributes (position, normal, color) for rendering.

D. Elevation Function

The function `getElevation(x,z)` performs the following steps:

1) Sample fractal noise at ($x \times$ `noiseScale`, $z \times$ `noiseScale`) for `octaves` with `persistence`, remapped to $[0, 1]$. Call this f.
2) Sample ridged noise at the same scaled coordinates. Call this r.
3) Blend: $e = f \times (1 -$ `ridgedMix`$) + r \times$ `ridgedMix`.
4) If `useTerrace` is true, quantize e into `terraceSteps` bands: $e \leftarrow \lfloor e \times$ `terraceSteps`$\rfloor /$ `terraceSteps`.

5) If `useIsland` is true, multiply e by $\max(0, 1 - \sqrt{x^2 + z^2})$.
6) Return e.

E. GUI Controls (ImGui)

All interactive controls reside in a single ImGui window labeled "Terrain Controls." Controls are organized into logical groups separated by `ImGui::Separator()` for readability:

- **Mode Toggle:** A checkbox labeled "Geometry Terrain" toggles the boolean `useGeo`. When checked, rendering uses the geometry-shader wireframe; when unchecked, it uses the flat shaded pipeline. Toggling this does *not* set `dirty`—no mesh rebuild is required, since the same vertex/index buffers are reused.
- **Noise Parameters:**
  - *Grid Size (int slider [16–256]):* Controls `gridSize`. Changing this sets `dirty = true` so that the vertex grid is regenerated to the new resolution.
  - *Noise Scale (float slider [0.1–10.0]):* Controls `noiseScale`. Changing this sets `dirty = true` to update elevation sampling.
  - *Octaves (int slider [1–8]):* Controls `octaves`. Changing sets `dirty = true`.
  - *Persistence (float slider [0.1–1.0]):* Controls `persistence`. Changing sets `dirty = true`.
  - *Ridged Blend (float slider [0.0–1.0]):* Controls `ridgedMix`. Changing sets `dirty = true`.
  - *Terrace (checkbox) + Terrace Steps (int slider [1–50]):* Controls `useTerrace` and `terraceSteps`. Checking "Terrace" or adjusting "Terrace Steps" sets `dirty = true`.
  - *Island Mask (checkbox):* Controls `useIsland`. Changing sets `dirty = true`.
- **Biome & Moisture:**
  - *Moisture Scale (float slider [0.1–10.0]):* Controls `moistureScale`. Changing sets `dirty = true`.
  - *Moisture Octaves (int slider [1–8]):* Controls `moistureOctaves`. Changing sets `dirty = true`.
  - *Moisture Persistence (float slider [0.1–1.0]):* Controls `moisturePersistence`. Changing sets `dirty = true`.
  - *Water Level (float slider [0.0–1.0]):* Controls `waterLevel`. Changing sets `dirty = true`, as it affects biome classification.
  - *Beach Level (float slider [0.0–1.0]):* Controls `beachLevel`. Changing sets `dirty = true`.
  - *Snow Level (float slider [0.0–1.0]):* Controls `snowLevel`. Changing sets `dirty = true`.
- **Weather Toggles:**
  - *Falling Snow (checkbox):* Controls `enableSnow`. When checked, new snowflakes will be spawned and updated each frame. This does not set `dirty`, since the terrain mesh itself is unaffected.
  - *Falling Rain (checkbox):* Controls `enableRain`. When checked, new raindrops will be spawned and updated each frame. This does not set `dirty`.

All sliders immediately set their corresponding variables "on release," so the viewer sees the terrain update as soon as they finish dragging. The `dirty` flag ensures that any change which affects the heightmap (grid resolution, noise parameters, terracing, island mask, or biome thresholds) triggers a regenerate of the mesh before the next draw. Because ImGui runs in the same frame as the rendering, the call to `buildTerrain()` happens before drawing the updated frame.

F. Particle Systems

Two lightweight particle systems (snow and rain) simulate precipitation. Initialization and update logic are as follows; the actual draw calls are described conceptually rather than shown as pseudocode.

Snow Update (Initialization & Simulation):

Each frame, up to 5 new snowflakes are spawned at random $(x, z) \in [-1, 1] \times [-1, 1]$ with initial height $y = 2$. The total number of snowflakes is capped at 5000.All existing snowflake positions are moved downward by a constant $\Delta y = 0.002$. If a snowflake's y position falls below the terrain elevation at (x, z), it is removed from the list. The updated list of 3D positions is uploaded to the GPU via the `snowVBO`. During rendering, the `particleShader` is bound, its uniform color is set to white, and the `snowVAO` (linked to `snowVBO`) is drawn as GL_POINTS with a fixed point size (e.g., 3.0), producing white "flakes" that fall and disappear on contact.

Rain Update (Initialization & Simulation):

- Each frame, up to 10 new raindrops are spawned at random $(x, z) \in [-1, 1] \times [-1, 1]$ with initial height $y = 2$. The total number of raindrops is capped at 5000.
- All existing raindrop positions are moved downward by $\Delta y = 0.01$.
- If a raindrop's y position falls below the terrain elevation at (x, z), it is removed from the list.
- The updated list of 3D positions is uploaded to the GPU via the `rainVBO`.
- During rendering, the `particleShader` is bound, its uniform color is set to blue, and the `rainVAO` (linked to `rainVBO`) is drawn as GL_POINTS with a fixed point size, producing blue "raindrops" that fall and disappear on contact.

In both cases, the simulation and buffer uploads occur on the CPU each frame, while the GPU draws colored points at the specified 3D coordinates. This approach keeps the particle systems simple and efficient.

G. Render Loop

The core application loop proceeds as follows:

1) **ImGui Frame Start:** Call `ImGui_ImplOpenGL3_NewFrame()`, `ImGui_ImplGlfw_NewFrame()`, and `ImGui::NewFrame()`. This also implicitly polls GLFW events (mouse, keyboard, etc.).
2) **Draw ImGui Controls:** Render the "Terrain Controls" window as described above. Any parameter changes set `dirty = true`.
3) **Rebuild Terrain if Needed:** If `dirty` is true, call `buildTerrain()` to regenerate the vertex/index buffers, then reset `dirty = false`.
4) **Update Particles:** If `enableSnow` is true, perform the snow update logic outlined above. If `enableRain` is true, perform the rain update logic outlined above.
5) **Compute Camera Matrices:** Calculate the camera's `eye` position from `yaw`, `pitch`, `distanceToCenter`. Then build the view matrix via `glm::lookAt(eye, (0,0,0), (0,1,0))` and the projection matrix via `glm::perspective(45°, WIDTH/HEIGHT, 0.1, 100)`. The combined $VP = P \times V$ is passed to all shaders.
6) **Clear Framebuffers:** Call `glViewport(0,0,WIDTH,HEIGHT)`, `glClearColor(0.6,0.8,1.0,1.0)`, and `glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT)`.
7) **Draw Terrain:** If `useGeo` is true, bind `geoProgram`, set its `uMVP` uniform to VP and `uTime` to the current time. Otherwise, bind `terrainShader`, set `uMVP = VP` and `uLightDir = (-1,1,-1)`. In either case, bind `terrainVAO` and issue `glDrawElements(GL_TRIANGLES, terrainIndexCount, GL_UNSIGNED_INT, 0)`.
8) **Draw Particles:** If `enableSnow` or `enableRain` is true, bind `particleShader` and set `uMVP = VP`. Then, if `enableSnow` is true, set `uColor = (1,1,1,1)`, bind `snowVAO`, and draw `glDrawArrays(GL_POINTS, 0, snowflakes.size())`. If `enableRain` is true, set `uColor = (0,0,1,1)`, bind `rainVAO`, and draw the raindrops likewise.
9) **Render ImGui Draw Data:** Call `ImGui::Render()` followed by `ImGui_ImplOpenGL3_RenderDrawData (ImGui::GetDrawData())` to composite the GUI on top of the 3D scene.
10) **Swap Buffers:** Call `glfwSwapBuffers(window)` to present the final image.

IV. Important Calculations & Foundations

A. Dot and Cross Products (Normals)

For triangle vertices A, B, C:

$$\mathbf{n} = \frac{(B-A)\times(C-A)}{\|(B-A)\times(C-A)\|}.$$

Each vertex normal is the normalized sum of all incident face normals.

B. Smoothstep Interpolation

The quintic fade function:

$$S(t) = 6t^5 - 15t^4 + 10t^3, \quad 0 \le t \le 1,$$

ensures smooth transitions with zero first and second derivatives at $t = 0$ and $t = 1$.

C. Fractal Summation (fBM)

$$\mathrm{fBM}(x,y) = \sum_{i=0}^{O-1} \big(\text{persistence}^i \times \text{noise}(x\cdot 2^i,\, y\cdot 2^i)\big).$$

Remapped to $[0,1]$ via $\times 0.5 + 0.5$.

D. Ridged Transformation

$$r_i = 1 - \big|\text{noise}(x\cdot 2^i,\, y\cdot 2^i)\big|, \quad \text{ridged} = \sum_{i=0}^{O-1} (\text{persistence}^i \cdot r_i).$$

E. Terracing

$$\text{terraced}(e) = \frac{\lfloor e \times \text{steps} \rfloor}{\text{steps}}.$$

F. Island Mask

$$\text{mask}(x,z) = 1 - \sqrt{x^2+z^2}, \quad e_{\text{island}} = e \times \text{mask}(x,z).$$

G. Geometry-Shader Ripple

$$\Delta y = A \sin\big(\text{frequency}\,\|(x,z)\| - uTime \times \text{speed}\big).$$

H. MVP Matrix

$$uMVP = P \times V,$$
$$V = \text{lookAt}\big(\text{eye},\,(0,0,0),\,(0,1,0)\big),$$
$$P = \text{perspective}\big(45^\circ,\, W/H,\, 0.1,\, 100\big).$$

V. Integration with ImGui

All ImGui controls reside in a single "Terrain Controls" window. Whenever the user adjusts any slider or checkbox that affects terrain shape (grid resolution, noise parameters, terracing, island mask, moisture scale, biome thresholds), a global flag `dirty` is set to `true`. At the beginning of each render loop iteration, the code checks:

```
if (dirty) {
   buildTerrain()
   dirty = false
}
```

This ensures the mesh buffers are regenerated before the 3D scene is drawn. The `dirty` flag thus decouples GUI interactions from geometry construction: any number of changes can occur within a single frame, but the terrain is rebuilt exactly once per frame if needed.

Moreover, mouse event callbacks begin with:

```
if (ImGui::GetIO().WantCaptureMouse) return;
```

so that clicking or dragging on any ImGui widget does not inadvertently rotate or zoom the camera. Only when ImGui indicates it is not capturing the mouse does the application apply changes to `yaw`, `pitch`, and `distanceToCenter`. This coordination ensures a smooth, intuitive experience: GUI manipulation never conflicts with camera control.

VI. Future Improvements

Several enhancements remain possible. First, exporting the map to a file (e.g., heightmap or mesh data) would allow use in external engines such as Unity; this entails writing vertex and index arrays to a standard format (PNG for heightmaps or OBJ/GLTF for meshes) once `buildTerrain()` has generated them. Second, camera translation could be added so the view can pan across the terrain rather than only rotate and zoom; implementing this requires adding keyboard or middle-mouse drag handlers to modify a camera origin offset before computing the view matrix. Third, more mathematical noise and distortion functions can further customize the landscape—such as Perlin-based erosion simulations, gradient-based caverns, or cellular (Worley) noise overlays. Moreover, the introduction of alternative wave functions in the geometry shader—replacing the current circular sine ripple with radial cosine, exponential falloff, or polygonal waveforms—by editing the vertex offset formula in `geoVert`. Each addition simply extends the existing `getElevation()` or shader code with new formulas, leveraging the same sampling and interpolation framework.

VII. Conclusion

The current work demonstrates a complete pipeline for procedural 3D terrain generation: a custom Perlin-noise library implementing gradient, fractal, ridged, terrace, and island-mask modifiers; mesh and normal construction; real-time biome coloring; and interaction via ImGui. A geometry-shader ripple and lightweight snow/rain particle systems show how shader-based effects can aid in visual variety. All core calculations such as gradient dot-products, quintic interpolation, fractal summation, and more, are implemented explicitly, providing a basis for further improvement and extension.

References

[1] K. Perlin, "An Image Synthesizer," *ACM SIGGRAPH Computer Graphics*, vol. 19, no. 3, pp. 287–296, 1985.

[2] T. J. Rose and A. G. Bakaoukas, "Algorithms and Approaches for Procedural Terrain Generation – A Brief Review of Current Techniques," 2016.

[3] W. C. Chow, "Fractal (fractional) Brownian motion," *WIREs Comput. Stat.*, vol. 3, no. 2, pp. 149–162, 2011.

[4] "Dear ImGui," https://github.com/ocornut/imgui, accessed 2025.

[5] O. E. Incic, "How to Generate Perlin Noise Textures in OpenGL," https://david.iancic.ro/how-to-generate-perlin-noise-textures-in-opengl/, accessed 2025.

[6] "Geometry Shader," *LearnOpenGL.com*, https://learnopengl.com/Advanced-OpenGL/Geometry-Shader, accessed 2025.

[7] D. Wolff, *OpenGL 4 Shading Language Cookbook*. Packt Publishing, 2018.

[8] J. Rodríguez, "GLSL Essentials," Packt Publishing, 2013.